

BÁO CÁO KỸ THUẬT - BÀI 2

PHÂN LỚP ĐƠN HÀNG GIAO TRỄ TRÊN OLIST

1. Bài toán và dữ liệu

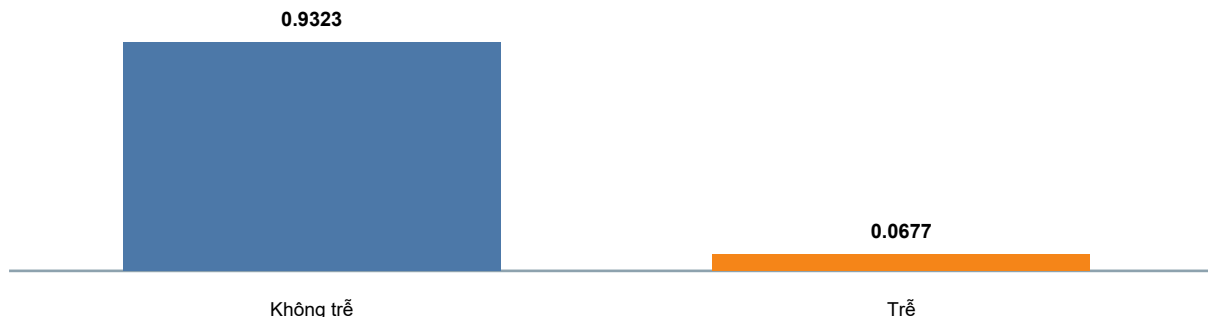
Mục tiêu là dự đoán một đơn hàng có giao trễ hay không. Biến mục tiêu `is_late` có hai lớp: 0 - không trễ và 1 - trễ. Nhân đã được tạo ở Bài 1 bằng quy tắc `delivery_delay_days > 0`. Do dùng số ngày nguyên, đơn giao sau hạn nhưng chênh chưa đủ 24 giờ có thể vẫn thuộc lớp 0; báo cáo giữ nguyên định nghĩa của notebook để kết quả khớp dữ liệu đầu vào.

Dữ liệu đọc từ `data/processed/olist/olist_classification.csv`, gồm 96.476 đơn và 11 cột, không có giá trị thiếu. `order_id` là khóa đối soát; `order_purchase_timestamp` là thời gian thô. Hai cột này và `target` không được đưa vào ma trận đặc trưng.

Đặc trưng	Ý nghĩa	Miền quan sát
<code>purchase_month</code>	Tháng đặt hàng	1-12
<code>purchase_dayofweek</code>	Thứ trong tuần, thứ Hai = 0	0-6
<code>purchase_hour</code>	Giờ đặt hàng	0-23
<code>order_item_count</code>	Số item trong đơn	1-21
<code>unique_product_count</code>	Số sản phẩm khác nhau	1-8
<code>unique_seller_count</code>	Số người bán khác nhau	1-5
<code>total_order_price</code>	Tổng giá sản phẩm	0,85-13.440
<code>total_freight_value</code>	Tổng phí vận chuyển	0-1.794,96

Dữ liệu mất cân bằng mạnh: 89.941 đơn không trễ (93,23%) và 6.535 đơn trễ (6,77%). Baseline luôn đoán lớp không trễ đã đạt `accuracy 0,9323` nhưng `recall` và `F1` của lớp trễ bằng 0. Vì vậy `accuracy` chỉ được báo cáo kèm `precision`, `recall`, `F1`, ma trận nhầm lẫn và `ROC-AUC`.

Hình 1. Tỷ lệ hai lớp



Số liệu trên đầu cột là tỷ lệ của từng lớp.

2. Thiết kế thí nghiệm

2.1. Chia dữ liệu và tiền xử lý

Notebook dùng `train_test_split` với `test_size=0,2`, `random_state=42` và `stratify=y`. Tập train có 77.180 đơn, tập test có 19.296 đơn. Tỷ lệ lớp trẻ ở cả hai tập đều khoảng 6,77%. Tập test chỉ dùng để đánh giá sau khi GridSearchCV chọn cấu hình trên train.

Tám đặc trưng đều là số. Pipeline vẫn khai báo hai nhánh: nhánh số điền median, nhánh hạng mục điền mode và one-hot để mã nguồn không lỗi nếu về sau bổ sung biến hạng mục. Dữ liệu hiện không thiếu nên imputer không thay đổi giá trị. StandardScaler chỉ dùng cho Logistic Regression và kNN; Decision Tree không cần scale vì chia bằng ngưỡng trên từng biến.

Toàn bộ imputer, encoder và scaler nằm trong Pipeline. Khi GridSearchCV chạy, các bước này chỉ fit trên phần train của từng fold, tránh lấy thông kê từ fold validation hoặc test. Tuy nhiên cách chia ngẫu nhiên chưa kiểm tra độ trôi theo thời gian; một thí nghiệm bổ sung bằng temporal split sẽ phù hợp hơn nếu mục tiêu là dự báo đơn tương lai.

2.2. Thuật toán, giả định và siêu tham số

Mô hình	Cơ chế và giả định	Không gian siêu tham số	Cấu hình được chọn
Dummy	Luôn đoán lớp phổ biến; dùng làm mốc để phát hiện accuracy đánh lừa.	<code>most_frequent</code>	Không có tuning
Decision Tree	Chia dữ liệu bằng ngưỡng để tăng độ thuần. Không cần tuyến tính hoặc scale; dễ overfit nếu cây sâu.	<code>max_depth = 3, 5, 7, 10, None;</code> <code>min_samples_leaf = 50, 100, 300;</code> <code>class_weight = None, balanced</code>	<code>balanced; depth=3; leaf=50</code>
Logistic Regression	Xác suất được mô hình hóa qua log-odds tuyến tính. Cần scale; khó nắm bắt quan hệ phi tuyến nếu không tạo thêm biến.	<code>C = 0,1; 1; 10; class_weight = None, balanced</code>	<code>C=10; balanced</code>
kNN	Lớp được quyết định từ các điểm gần nhất. Phụ thuộc thang đo và mật độ lân cận.	<code>k = 3, 5, 7, 9, 11; weights = uniform, distance</code>	<code>k=3; distance</code>

GridSearchCV dùng StratifiedKFold 5 fold, `shuffle=True`, `random_state=42` và chọn theo F1 của lớp trẻ. Decision Tree thử 30 cấu hình, Logistic Regression 6 và kNN 10. kNN chỉ dùng mẫu train phân tầng 40.000 dòng để giảm thời gian; do đó điểm CV của kNN không hoàn toàn trực tiếp so sánh với hai mô hình dùng toàn bộ train.

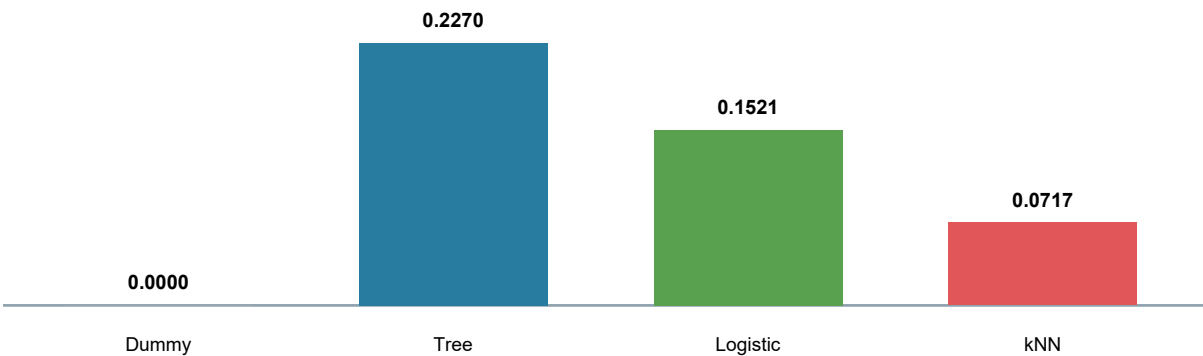
F1 tốt nhất trên CV: Decision Tree 0,2228; Logistic Regression 0,1502; kNN 0,0753. `class_weight='balanced'` được chọn cho Tree và Logistic, cho thấy việc tăng trọng số lớp thiểu số giúp mục tiêu F1 hơn cấu hình không cân bằng.

3. Kết quả trên tập test

Mô hình	Accuracy	Precision trễ	Recall trễ	F1 trễ	ROC-AUC
Dummy	0,9323	0,0000	0,0000	0,0000	0,5000
Decision Tree	0,7483	0,1433	0,5455	0,2270	0,6828
Logistic Regression	0,5627	0,0876	0,5792	0,1521	0,5749
kNN	0,9087	0,1151	0,0520	0,0717	0,5409

Decision Tree có F1 lớp trễ cao nhất và ROC-AUC cao nhất. Logistic Regression có recall cao nhất, nhưng precision thấp vì dự đoán quá nhiều đơn là trễ. kNN giữ accuracy cao nhưng bỏ sót gần như toàn bộ lớp trễ. Dummy chứng minh rằng accuracy 93,23% không đồng nghĩa mô hình hữu ích.

Hình 2. F1 của lớp giao trễ trên tập test



Số liệu trên đầu cột là F1 của lớp is_late=True.

3.1. Ma trận nhầm lẫn

Mô hình	True Negative	False Positive	False Negative	True Positive
Dummy	17989	0	1307	0
Decision Tree	13727	4262	594	713
Logistic Regression	10101	7888	550	757
kNN	17466	523	1239	68

Decision Tree phát hiện đúng 713/1.307 đơn trễ và bỏ sót 594 đơn, nhưng tạo 4.262 cảnh báo giả. Logistic phát hiện thêm 44 đơn trễ so với Tree nhưng tạo thêm 3.626 cảnh báo giả. kNN chỉ phát hiện 68 đơn trễ và bỏ sót 1.239 đơn. Nếu chi phí bỏ sót cao hơn chi phí kiểm tra cảnh báo, có thể ưu tiên recall và điều chỉnh threshold; notebook hiện dùng threshold mặc định 0,5 nên chưa đánh giá trade-off này.

ROC-AUC đo khả năng xếp hạng mẫu trễ cao hơn mẫu không trễ trên nhiều threshold. AUC 0,6828 của Tree cho thấy khả năng phân biệt ở mức hạn chế, chưa đủ để kết luận mô hình mạnh. Với lớp dương chỉ 6,77%, Precision-Recall AUC sẽ cung cấp thêm góc nhìn phù hợp nhưng notebook chưa tính nên báo cáo không tự thêm số liệu.

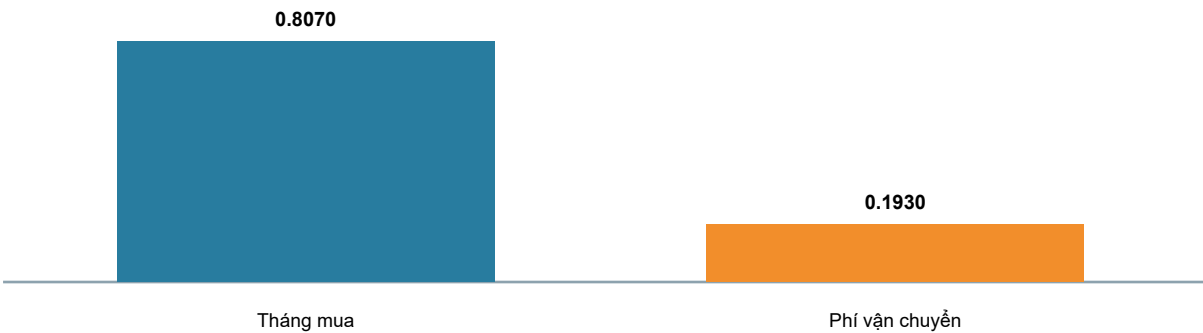
4. Nhận xét kỹ thuật và mức độ đáp ứng

4.1. Mô hình được chọn

Trong các cấu hình đã thử, Decision Tree cân bằng, max_depth=3, min_samples_leaf=50 là lựa chọn tốt nhất theo tiêu chí đã đặt ra: F1 lớp trẻ 0,2270. Cây nông giúp hạn chế overfit và dễ kiểm tra hơn. Lựa chọn này không có nghĩa mô hình đã đủ tốt để triển khai, vì precision 0,1433 cho biết phần lớn cảnh báo trẻ là false positive.

Feature importance của cây chỉ phân bổ cho purchase_month = 0,8070 và total_freight_value = 0,1930; sáu biến còn lại bằng 0 trong cây sâu 3. Đây là mức độ sử dụng biến để chia nút, không chứng minh tháng đặt hàng hay phí vận chuyển gây ra giao trễ. Kết quả có thể thay đổi theo mẫu, định nghĩa nhãn và siêu tham số.

Hình 3. Feature importance của Decision Tree



Con số trên đầu cột là tổng mức giảm impurity chuẩn hóa của từng đặc trưng được cây sử dụng.

4.2. Điểm mạnh, điểm yếu và cải tiến

Nội dung	Đánh giá
Điểm mạnh	Pipeline tránh rò rỉ từ impute/scale; có baseline; so sánh ba thuật toán; tuning bằng CV; dùng đầy đủ accuracy, precision, recall, F1, confusion matrix và ROC-AUC.
Hạn chế dữ liệu	Chỉ có đặc trưng thời điểm mua và quy mô/giá trị đơn. Chưa có khoảng cách giao, bang người bán/khách, loại sản phẩm, mùa cao điểm hoặc năng lực vận chuyển.
Hạn chế thiết kế	Random split có thể lạc quan khi dữ liệu thay đổi theo thời gian. purchase_month được xem như số tháng dù tháng 12 gần tháng 1. KNN chỉ học trên 40.000 dòng.
Cải tiến hợp lý	Thử temporal split; mã hóa tháng/giờ theo sin-cos; tối ưu threshold theo chi phí; bổ sung PR-AUC; kiểm tra calibration; thêm đặc trưng chỉ có tại thời điểm đặt hàng.
Không được kết luận	Feature importance không phải quan hệ nhân quả. Accuracy cao của KNN/Dummy không chứng minh phát hiện tốt đơn trễ.

4.3. Đối chiếu yêu cầu giáo viên

Phần Olist đáp ứng các yêu cầu: xác định target và mức cân bằng; tách train/test có stratify; tiền xử lý trong Pipeline; nêu cơ chế và giả định; so sánh ít nhất hai thuật toán; thử siêu tham số có phương pháp; báo cáo accuracy, precision, recall, F1, ma trận nhầm lẫn và ROC-AUC; nhận xét ngắn ở mức kỹ thuật.

Theo ma trận Bài 1, US Accidents cũng được đánh dấu cho phân lớp. Tuy nhiên phan-lop.ipynb hiện tại chỉ chứa Olist. Vì vậy báo cáo này không chèn lại kết quả US Accidents từ một phiên bản cũ không khớp notebook. Nếu đây là báo cáo chung của cả nhóm, cần bổ sung phần US Accidents vào chính notebook, chạy lại và sau đó ghép kết quả vào báo cáo trước khi nộp.